

Documentation

Introduction

Commercial banks receive a lot of applications for credit cards. Many of them get rejected for many reasons, like high loan balances, low-income levels, or too many inquiries on an individual's credit report, for example. Manually analyzing these applications is mundane, error-prone, and time-consuming (and time is money!). Fortunately, this task can be automated with the power of machine learning. In this project, we will build a model to predict credit card approval using machine learning techniques, so these issues can be solved.

Problem statement

Banks have to develop a machine learning model within 3 months to instantly predict credit card approval instead of waiting for a minimum of 7 days.

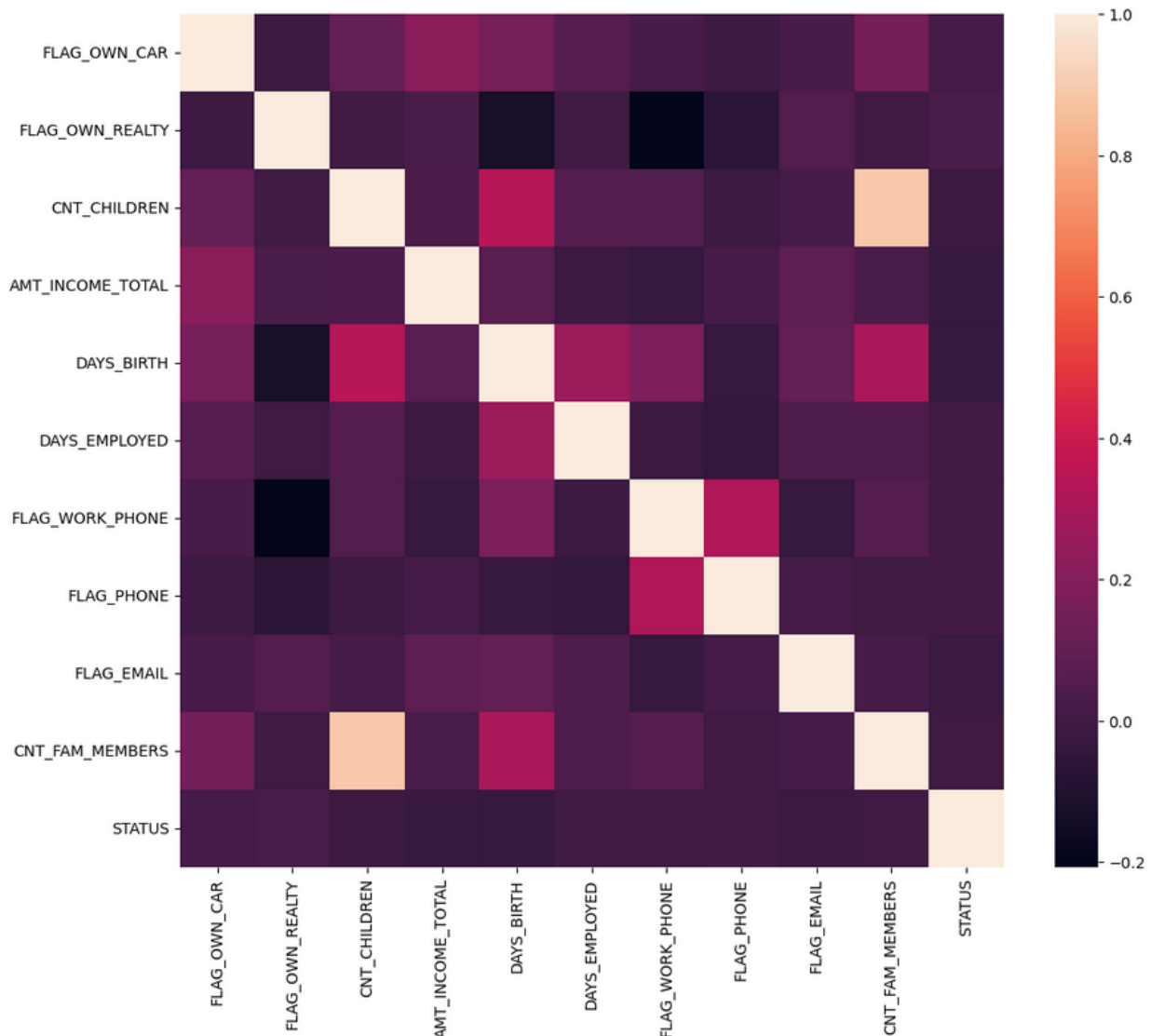
Data Wrangling

At first, we had two dataframes; application_details and credit_details that are connected by IDs. In credit_details, column MONTHS_BALANCE is the month of the extracted data is the starting point, backwards, 0 is the current month, -1 is the previous month, and so on. And column STATUS is 0: 1-29 days past due 1: 30-59 days past due 2: 60-89 days overdue 3: 90-119 days overdue 4: 120-149 days overdue, Overdue or bad debts, write-offs for more than 150 days C: paid off that month X: No loan for the month. To separately identify the count of X,C,0,1,2,3,4 & 5 for each ids, we add separate columns for that and deleted the existed two columns. Also we add a new column STATUS, which specify whether a customer is good (having no loans, paid all loans and having just 1-29 days past due.) or bad (having loan dues 30 days or more). We then generated a new dataframe by combining both the existing dataframes, df_applicant. It had 438557 rows and 27 columns. We dropped many rows having missing values for 'No.of X', 'No.of C' 'No.of 0', 'No.of 1', 'No.of 2', 'No.of 3', 'No.of 4' and 'No.of 5' as we donot have info whether or not these customers have loans. Also we dropped a column Occupation type as it have many missing values and so cannot consider to predict credit card approval. We also changed the datatypes of some columns as needed. FLAG_OWN_CAR and FLAG_OWN_REALTY had data as Y or N. We changed that to 1 or 0, to match with the other columns with FLAG.

No duplicates rows were found. No inconsistency found for categorical variable. Checked 'days_birth' is valid, considering the minimum age to open a bank account as 18years = 6570 days. DAYS_EMPLOYED have some outliers as shown in the histogram, which is surely incorrect. It can be happened by mistake. We replaced unrealistic values with the calculated mean of those valid values. Plotting the distribution of Customer STATUS, we can understand there are less than 5000 bad customers and more than 30,000 good customers. After all these cleaning process we have now 36457 rows and 26 columns in the dataframe.

Exploratory data Analysis

At this Exploratory Data Analysis step of the project, I explored the data, visualize the relationships between variables and find out patterns and emerging trends. Our target variable was STATUS. We can see Good customers are more than 30,000 (32166) and Bad customers are less than 5000(4291) by plotting the histogram. The 'STATUS' column was updated, 'GOOD CUSTOMER' to 1, 'BAD CUSTOMER' to 0, so that it will be easy to do all calculations. We then started to check the correlations between the variables. We use `.corr()` method, plotted a heatmap, boxplots, multiple_stacked_bars and categorical_proportion_heatmaps.



When used `.corr()` method and heatmap most of the features were found not very much related with others. Plotted box plots to find the relationship between STATUS column with other numeric columns. It showed the status of customer is not much affected by 'AMT_INCOME_TOTAL', 'DAYS_BIRTH', 'DAYS_EMPLOYED', 'CNT_CHILDREN' and 'CNT_FAM_MEMBERS' columns. Plotted multiple_stacked_bars to find the relationship between STATUS column with other categorical columns(having 2 types of category). It showed that the proportion for different types of different categories is almost similar for both good and bad customers. So these columns do not show influence in STATUS column. We then plotted

categorical_proportion_heatmaps to find the relationship between STATUS column with other categorical columns(having more than 4 types of category). These plots also shows that the proportion of different types of different category do not influence the STATUS column. So we were not able to find any relationship between STATUS column with other features. Finally, the no. of columns was reduced from 26 to 17.

Model Preprocessing with feature engineering

During Preprocessing_and_training, at first there were 17 columns. The dataset was cleaned, transformed, encoded(resulting in adding 10 more columns), and split into training and testing subsets. Feature scaling was applied following the split to prevent data leakage. The resulting datasets are suitable for the modeling phase.

Algorithms used to build the model with evaluation metric

This is a classification problem, in supervised learning. Here we have used the following classification models: Logistic Regression, K-Nearest Neighbor (KNN), Decision Trees, Random Forest & Gradient Boost.

Evaluating the performance of a model by training and testing on the same dataset can lead to the overfitting. Hence the model evaluation is based on splitting the dataset into train and validation set. But the performance of the prediction result depends upon the random choice of the pair of (train,validation) set. Inorder to overcome that, the Cross-Validation procedure is used where under the k-fold CV approach, the training set is split into k smaller sets, where a model is trained using k-1 of the folds as training data and the model is validated on the remaining part. Since this is an imbalanced classification problem, we focuses on f1-score as a metrics.

The modeling stage evaluated five candidate models using cross-validated F1 scores on the training data and F1 scores on the test set. Gradient Boosting and Random Forest achieved the highest performance.

Best model: Gradient Boosting-

- Highest CV F1-macro

- Best generalization on original test set

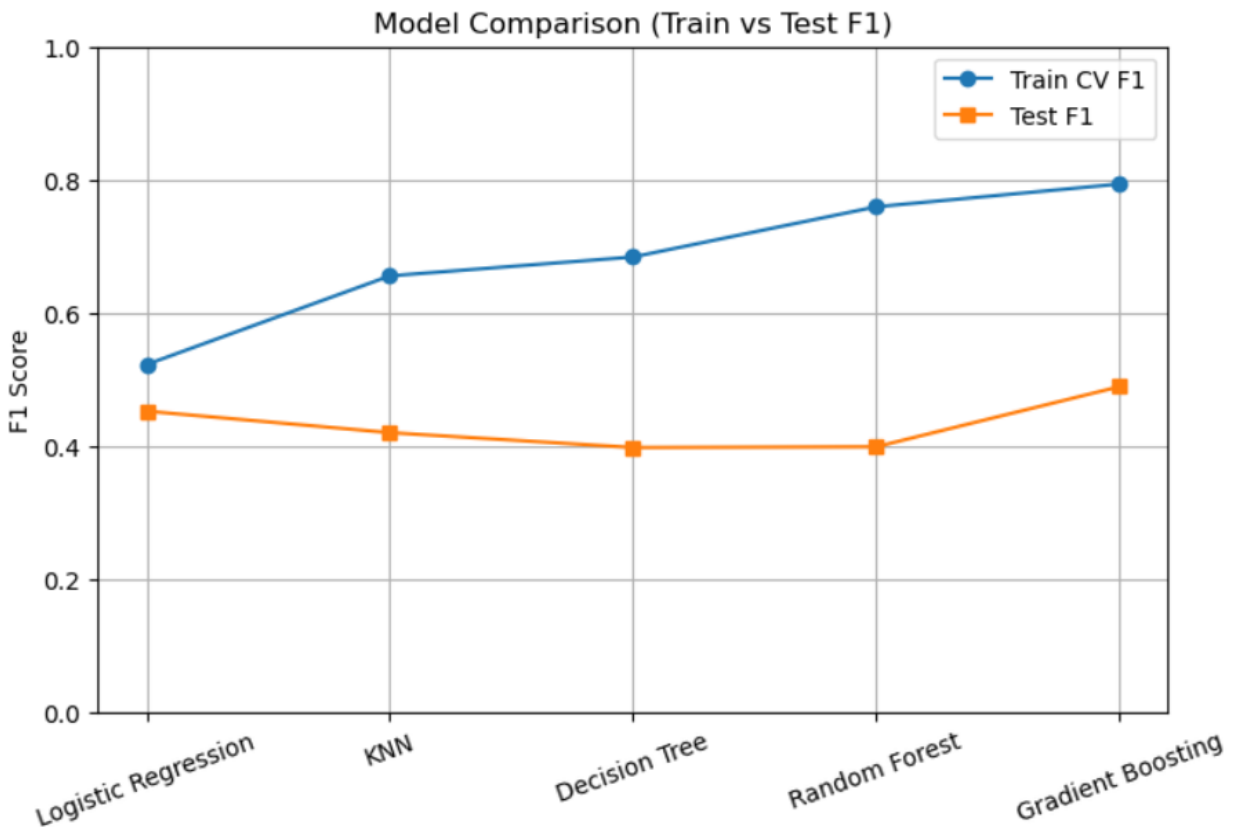
- Stable across folds (low CV std)

Second-best model: Random Forest-

- Still strong ensemble

- Slightly worse F1-macro, especially on minority class

Other models (Logistic Regression, KNN, Decision Tree) either underfit or fail to capture minority class well, so they are not recommended for final DSM deployment. After performing hyperparameter tuning, the final model were selected.



Winning model

Among all evaluated models, Gradient Boosting demonstrated the best balance between minority and majority class performance based on F1-macro. Hence, it was selected as the final model.

Recommendations

- With this model, when new credit card applications occur, bank can instantly predict whether the applicants qualify for credit card by giving the applicant details to the model.
- Business improves as bank can instantly predict credit card approval to the right customers.
- Bank can easily find out those who are ineligible for credit card, so they can prevent any loss happening to the business by not approving credit card for them.

Limitations

- Class imbalance affects minority class prediction.
- ROC-AUC is low due to imbalance.
- SMOTE may not reflect real-world distributions.

Conclusion

The Gradient Boost model identifies total income, age, and employment duration as the most influential predictors of STATUS. This indicates that financial stability and employment history play a key role in classifying customers as 'Good' or 'Bad'.

Future scope of work

- Try XGBoost / LightGBM,
- Cost-sensitive learning,
- More feature engineering,
- External data integration